Classification bayésienne supervisée de pixels : analyse des espaces colorimétriques RGB, HSV et Y′CbCr

1st Jair Anderson Vasquez Torres
Ingénieur Degree Programme STIC
ENSTA Paris
Paris, France
jair-anderson.vasquez@ensta.fr

2nd Santiago Florido Gomez
Ingénieur Degree Programme STIC
ENSTA Paris
Paris, France
santiago.florido@ensta-paris.fr

***Abstract*—Ce travail présente un cadre de classification bayésienne supervisée appliqué à la segmentation de pixels en imagerie couleur. L'article formalise les critères de décision Maximum A Posteriori (MAP) et Maximum Vraisemblance (ML), puis étudie le rôle du choix de l'espace d'observation dans la séparabilité statistique des classes. Trois représentations colorimétriques sont analysées : RGB, HSV et Y′CbCr. Nous décrivons leurs transformations, leurs propriétés pertinentes pour l'inférence probabiliste, et l'extraction opérationnelle de leurs composantes au moyen d'une implémentation Python/OpenCV. Les visualisations et canaux exportés (`R,G,B,H,S,V,Y′,Cb,Cr`) fournissent une base reproductible pour la construction de modèles bayésiens de classification de pixels et la comparaison de performances entre espaces colorimétriques.**

***Index Terms*—Classification bayésienne, Maximum A Posteriori, Maximum Vraisemblance, segmentation de pixels, RGB, HSV, Y′CbCr, OpenCV.**

I. Convolutional neural networks et CIFAR-10

Dans le contexte de la classification d'images, les réseaux de neurones convolutionnels se sont consolidés comme l'une des architectures les plus adaptées à l'analyse des données visuelles, car ils permettent l'extraction automatique de caractéristiques spatiales hiérarchiques à partir des données d'entrée. Un réseau neuronal implémenté peut être modélisé comme une application composée

$$f_\theta : \mathbb{R}^{H\times W\times C} \to \mathbb{R}^K, \tag{1}$$

où H et W sont la hauteur et la largeur de l'image, C est le nombre de canaux, K est le nombre de classes et θ désigne l'ensemble des paramètres entraînables.

Cette architecture remplace la connexion dense d'un réseau totalement connecté par une structure à champs réceptifs locaux, partage de poids et extraction hiérarchique de caractéristiques. Une CNN est organisée en plusieurs étapes, et chaque étape contient essentiellement une couche de filtres, une non-linéarité et une opération de réduction ou de *pooling* sur des cartes de caractéristiques [2].

Soit $a^{(0)} = x$ l'entrée. Dans une couche convolutionnelle l, si l'entrée possède C_{l-1} cartes et la sortie possède C_l cartes, l'opération peut s'écrire

$$z^{(l)}_{i,j,m} = b^{(l)}_m + \sum_{c=1}^{C_{l-1}} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K^{(l)}_{u,v,c,m}\, a^{(l-1)}_{i+u,\, j+v,\, c}, \tag{2}$$

et, après la convolution, une non-linéarité ponctuelle est appliquée :

$$a^{(l)}_{i,j,m} = \phi\Big(z^{(l)}_{i,j,m}\Big). \tag{3}$$

La dimension spatiale de la sortie d'une convolution avec pas (*stride*) s et remplissage (*padding*) p est donnée par

$$H_l = \left\lfloor \frac{H_{l-1} + 2p_h - k_h}{s_h} \right\rfloor + 1, \qquad W_l = \left\lfloor \frac{W_{l-1} + 2p_w - k_w}{s_w} \right\rfloor + 1. \tag{4}$$

Cela permet de contrôler la résolution spatiale à mesure que le signal traverse le réseau.

Si un réseau possède plusieurs couches, le champ réceptif effectif d'un neurone dans les couches profondes croît avec la profondeur. Une manière utile de le modéliser est donnée par la récurrence

$$R_1 = k_1, \qquad R_l = R_{l-1} + (k_l - 1)\prod_{t=1}^{l-1} s_t, \tag{5}$$

où R_l est la taille du champ réceptif effectif dans la couche l.

L'un des principaux avantages quantitatifs de la convolution est son efficacité paramétrique. Si une couche convolutionnelle possède C_{in} canaux d'entrée, C_{out} canaux de sortie et des filtres de taille $k_h \times k_w$, alors le nombre de paramètres est

$$N^{\text{conv}}_{\text{params}} = (k_h k_w C_{\text{in}} + 1)\, C_{\text{out}}. \tag{6}$$

En revanche, une couche dense reliant une entrée de taille n_{in} à une sortie de taille n_{out} possède

$$N^{\text{fc}}_{\text{params}} = (n_{\text{in}} + 1)\, n_{\text{out}}. \tag{7}$$

Cette différence constitue un élément significatif dans la classification automatique d'images, puisqu'une image d'entrée peut contenir des milliers de variables, tandis que

la convolution exploite leurs caractéristiques structurelles en évitant une croissance explosive du nombre de poids.

Dans l'étape finale de classification, la représentation obtenue par les couches convolutionnelles est aplatie ou agrégée globalement afin de produire un vecteur $h \in \mathbb{R}^d$. Ensuite, une couche affine calcule

$$z = Wh + b, \qquad z \in \mathbb{R}^K, \tag{8}$$

et la probabilité estimée pour la classe k est obtenue au moyen de la fonction *softmax* :

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}. \tag{9}$$

[2]

Il est proposé d'utiliser la base de données CIFAR-10, largement employée dans l'évaluation des algorithmes de classification d'images, qui contient $60\,000$ images RGB de 32×32 pixels réparties en 10 classes, avec $6\,000$ images par classe. La partition standard comprend $50\,000$ images pour l'entraînement et $10\,000$ pour le test ; de plus, l'ensemble de test contient exactement $1\,000$ images par classe et l'ensemble d'entraînement en contient $5\,000$ par classe. Les classes sont *airplane*, *automobile*, *bird*, *cat*, *deer*, *dog*, *frog*, *horse*, *ship* et *truck* [3], ce qui représente un scénario adéquat pour l'analyse des éléments fondamentaux de l'apprentissage profond avec des exigences computationnelles modérées.

Comme une partie de l'objectif de recherche consiste également à étudier de manière contrôlée l'influence de la préparation des données, de la définition de l'architecture, de la sélection des hyperparamètres et de la définition de la dynamique d'apprentissage sur la performance idéale du modèle implémenté, ce cadre expérimental se révèle particulièrement pertinent.

II. Structuration des données

A. *training, validation, test*

La division de la base de données constitue un élément méthodologique propre à l'apprentissage supervisé, de manière à isoler l'information destinée à l'ajustement des paramètres, à la sélection du modèle et à l'évaluation du modèle entraîné. L'ensemble d'entraînement est utilisé pour l'estimation des paramètres du réseau, l'ensemble de validation a pour objectif de mesurer la généralisation du modèle sans intervenir directement dans les actualisations des paramètres, et, enfin, l'ensemble de test sert à obtenir une estimation indépendante et non biaisée de la performance du modèle sur des données qu'il n'a pas vues.

L'objectif de cette séparation est d'éviter la fuite d'information, qui peut se produire lorsque les données d'évaluation du modèle ont été utilisées dans l'estimation des paramètres et dans la définition de celui-ci, afin de pouvoir disposer de véritables mesures de la capacité de généralisation du modèle entraîné.

B. *fonction standardize*

La fonction `standardize` peut être comprise comme une standardisation spatiale réalisée image par image et canal par canal. Pour un ensemble d'images représenté par un tenseur $x \in \mathbb{R}^{N \times H \times W \times C}$, la fonction calcule, pour chaque image n et pour chaque canal c, la moyenne $\mu_{n,c}$ et l'écart-type $\sigma_{n,c}$ sur les dimensions spatiales (H, W), puis transforme chaque pixel selon l'expression (10) :

$$x'_{n,i,j,c} = \frac{x_{n,i,j,c} - \mu_{n,c}}{\sigma_{n,c}}. \tag{10}$$

L'implication de cette implémentation est que chaque canal de chaque image est recentré autour de zéro et redimensionné de sorte que la dispersion soit aussi proche que possible d'une dispersion unitaire. Cet effet d'homogénéisation de l'entrée, en termes de son échelle numérique, facilite le processus d'optimisation et peut être utile pour accroître la stabilité de l'entraînement, car, en réduisant les différences globales d'intensité et de contraste entre les images, la standardisation diminue la variabilité non informative mais présente dans les données d'entrée, de manière à favoriser l'apprentissage par le réseau des motifs structurels les plus pertinents.

III. Architecture du réseau

Dans cette architecture, on définit un réseau de neurones convolutionnel séquentiel composé d'une étape d'extraction de caractéristiques locales au moyen de la convolution, suivie de réductions dimensionnelles par *max pooling*. La couche d'entrée est définie pour un tenseur d'entrée $32 \times 32 \times 3$, où 32 est la hauteur, 32 la largeur et 3 le nombre de canaux de couleur (R, G, B). Cette couche n'apprend pas de paramètres ; elle fixe uniquement la forme attendue des données d'entrée du modèle [6].

Ensuite, une première couche convolutionnelle est définie pour la première extraction des caractéristiques. La couche apprend 8 filtres distincts, ce qui implique qu'à sa sortie elle aura 8 cartes de caractéristiques. Une dimension de noyau $(3, 3)$ est définie, et l'argument `padding="same"` est incorporé dans la définition, ce qui permet de préserver la résolution spatiale de l'image, de sorte que la sortie conserve une hauteur et une largeur égales à celles de l'entrée. Par conséquent, la sortie de cette couche a la taille

$$32 \times 32 \times 8. \tag{11}$$

Une couche de *dropout* est ajoutée, laquelle remplit la fonction de mécanisme de régularisation durant l'entraînement en annulant aléatoirement une fraction des activations des entrées. Néanmoins, puisque la valeur du *dropout* est fixée par défaut à zéro, la couche ne modifie ni l'information ni la dimension du tenseur. Sa sortie demeure

$$32 \times 32 \times 8. \tag{12}$$

On ajoute une couche de *max pooling* qui a un *pool size* $(2, 2)$, ce qui implique qu'elle réduit la résolution spatiale en

prenant la valeur maximale dans chaque fenêtre locale 2×2. De cette manière, la sortie de cette couche est

$$16 \times 16 \times 8. \tag{13}$$

On ajoute ensuite une couche qui convertit le vecteur tridimensionnel en un vecteur unidimensionnel de dimension

$$16 \cdot 16 \cdot 8 = 2048, \tag{14}$$

puis une transformation *fully connected* est implémentée au moyen d'une couche dense vers un espace latent de 64 neurones :

$$\mathbf{h} = \mathrm{ReLU}(W\mathbf{x} + \mathbf{b}). \tag{15}$$

De sorte que la sortie de cette couche est de taille 64, on ajoute une deuxième couche de *dropout* dans laquelle le *rate* est fixé à 0, pour finalement inclure une couche dense supplémentaire vers un espace latent de dimension 10. Dans cette occasion, l'activation *softmax* transforme les scores en une distribution de probabilité :

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{10} e^{z_j}}, \qquad k = 1, \ldots, 10. \tag{16}$$

Par conséquent, chaque composante de la sortie appartient à l'intervalle $[0, 1]$ et la somme de toutes est égale à 1. La classe prédite est celle associée à l'indice ayant la probabilité la plus élevée :

$$\hat{c} = \arg\max_k \hat{y}_k. \tag{17}$$

Enfin, il y a une dernière couche de *dropout* avec un taux égal à 0.

Les paramètres entraînables sont ceux que le modèle met à jour durant l'entraînement au moyen de la rétropropagation et de l'optimisation, lesquels, dans le cas de la structure proposée, sont incorporés dans les couches convolutionnelles et denses, puisque des couches telles que `Input`, `Dropout`, `MaxPooling2D` et `Flatten` n'incorporent pas de paramètres entraînables dans leur définition. Dans le cas de la couche `Conv2D`,

$$N_{\text{params}} = (k_h \cdot k_w \cdot C_{\text{in}} + 1) \cdot C_{\text{out}}, \tag{18}$$

où k_h et k_w sont les dimensions du filtre, C_{in} le nombre de canaux d'entrée, et C_{out} le nombre de filtres de sortie. Ici,

$$k_h = 3, \qquad k_w = 3, \qquad C_{\text{in}} = 3, \qquad C_{\text{out}} = 8. \tag{19}$$

Alors,

$$N_{\text{params}} = (3 \cdot 3 \cdot 3 + 1) \cdot 8 = (27 + 1) \cdot 8 = 224. \tag{20}$$

C'est-à-dire que cette couche possède 224 paramètres entraînables, y compris le biais [4].

Pour sa part, dans la couche dense avec biais,

$$N_{\text{params}} = (n_{\text{in}} + 1) \cdot n_{\text{out}}, \tag{21}$$

où n_{in} est le nombre d'entrées et n_{out} le nombre de neurones de sortie. Dans le cas de la structure proposée, on a deux couches denses :

$$N_{\text{params}} = (2048 + 1) \cdot 64 = 2049 \cdot 64 = 131136, \tag{22}$$

et

$$N_{\text{params}} = (64 + 1) \cdot 10 = 650. \tag{23}$$

Ainsi, le nombre total de paramètres entraînables est

$$224 + 131136 + 650 = 132010. \tag{24}$$

IV. Apprentissage

Un *batch* est le sous-ensemble d'échantillons d'entraînement qui sont traités conjointement par le modèle à chaque itération, et le *batch size* est le nombre d'échantillons utilisés dans la mise à jour des paramètres au moyen du calcul du gradient, c'est-à-dire qu'il s'agit du lot utilisé dans la mise à jour des paramètres [5].

Un *step* est défini comme une itération de l'entraînement qui est associée au traitement d'un *batch*. Dans Keras, `steps_per_epoch` est utilisé comme le nombre total de *steps* (lots d'échantillons) avant de considérer une époque comme terminée, et TensorFlow indique en outre que la fonction `train_step` est appelée par `fit()` pour chaque lot de données. De cette manière, on peut considérer un *step* comme le traitement d'un lot, son entraînement, ainsi que la mise à jour résultante de ce lot [5].

Finalement, une époque est un passage complet effectué sur l'ensemble des données d'entraînement ; il s'agit donc du passage par l'ensemble des *batches* nécessaires pour parcourir toute la base d'entraînement, à moins qu'un critère comme `steps_per_epoch` ne soit imposé [5].

Les résultats obtenus suggèrent que le temps par époque diminue à mesure que la taille du *batch* augmente, puisque le modèle traite davantage d'échantillons à chacun des *steps*, ce qui fait que le nombre de *steps* nécessaires pour parcourir l'ensemble du jeu d'entraînement est plus faible. La réduction est beaucoup plus significative lorsqu'on passe de *batches* de très petite taille, en comparaison avec les variations observées pour des tailles intermédiaires, ce qui montre que le temps moyen par époque ne diminue pas linéairement lorsque l'on modifie la taille du *batch*. Cela s'explique principalement par le fait que le temps par époque ne dépend pas uniquement du nombre de *steps*, mais aussi du temps de chacun de ces *steps*. Ainsi, à mesure que la taille du *batch* augmente, le nombre de *steps* nécessaires pour parcourir la base d'entraînement diminue, mais le coût de chacun des pas augmente, puisque le modèle doit effectuer davantage d'opérations de calcul et de mémoire, ce qui est la cause de cette dynamique non linéaire.

En conséquence, le bénéfice initial est important lorsque le matériel est sous-utilisé, mais il devient progressivement plus faible à mesure que le traitement s'approche de ses limites d'efficacité et de mémoire, comme le montre la figure 1.

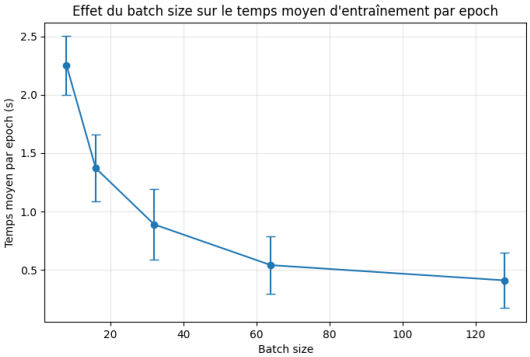

Fig. 1. Temps moyen d'entraînement par époque en fonction de la taille du *batch*.

REFERENCES

[1] Keras, "Input object," Keras 3 API documentation, accessed Mar. 8, 2026. [Online]. Available: https://keras.io/api/layers/core_layers/input/

[2] Y. LeCun, K. Kavukcuoglu, and C. Farabet, "Convolutional Networks and Applications in Vision," presented at ISCAS 2010, accessed Mar. 8, 2026. [Online]. Available: https://cseweb.ucsd.edu/~gary/cs200/s12/lecun-iscas-10.pdf

[3] A. Krizhevsky, "CIFAR-10 and CIFAR-100 datasets," accessed Mar. 8, 2026. [Online]. Available: https://www.cs.toronto.edu/~kriz/cifar.html

[4] Keras, "Conv2D layer," Keras 3 API documentation, accessed Mar. 8, 2026. [Online]. Available: https://keras.io/api/layers/convolution_layers/convolution2d/

[5] Keras, "Model training APIs," Keras 3 API documentation, accessed Mar. 8, 2026. [Online]. Available: https://keras.io/api/models/model_training_apis/

[6] Keras, "Input object," Keras 3 API documentation, accessed Mar. 8, 2026. [Online]. Available: https://keras.io/api/layers/core_layers/input/